

Laboratorio 1 – GPIO

Tecnologías de Microprocesamiento

Sergio Romero, Anthony Bonino, Santiago Moizo

Ingeniería Mecatrónica – UTEC

Semestre IV

Docentes: Jesús Martínez / Marcelo Díaz

Resumen—En este laboratorio se desarrollan cuatro actividades utilizando el microcontrolador ATmega328P programado en lenguaje ensamblador. Los problemas propuestos abarcan el control de una lavadora automática mediante una máquina de estados, un contador digital con display de siete segmentos, un sistema de selección y ejecución de ocho secuencias luminosas con ocho LEDs y una unidad aritmético-lógica (ALU) de 4 bits.

A lo largo del trabajo se aplican conceptos de configuración de puertos GPIO, manejo de registros, saltos condicionales, operaciones aritméticas y lógicas, retardos por software, lectura de entradas digitales y control directo de periféricos.

En la implementación final de la ALU se utilizaron interruptores tipo DIP switch para establecer los operandos y el código de selección. El montaje fue reorganizado para utilizar directamente los GPIO disponibles del Arduino UNO, sin registro de desplazamiento externo. Las entradas se implementaron con resistencias *pull-up* internas y se invirtieron por software para que DIP ON = 1 y DIP OFF = 0. El resultado y las banderas se visualizan mediante LEDs conectados directamente al ATmega328P.

Palabras clave—ATmega328P, AVR, ensamblador, GPIO, máquina de estados, display de siete segmentos, LEDs, ALU, DIP switch.

I. INTRODUCCIÓN

El objetivo general del laboratorio es familiarizarse con los aspectos fundamentales del microcontrolador ATmega328P y con la programación en lenguaje ensamblador para arquitectura AVR.

Para ello se utilizan registros de propósito general, el registro de estado SREG, puertos de entrada/salida, instrucciones aritméticas y lógicas, instrucciones de salto y rutinas de retardo implementadas mediante software.

La metodología de trabajo consiste en analizar cada problema, proponer una estructura lógica, implementar el programa en ensamblador, simular su funcionamiento y finalmente verificar su comportamiento mediante una implementación física.

II. OBJETIVOS

II-A. Objetivo general

Diseñar e implementar soluciones de control digital utilizando el ATmega328P, aplicando programación en ensamblador y manejo directo de entradas y salidas.

II-B. Objetivos específicos

- Configurar puertos GPIO del ATmega328P como entradas y salidas.
- Utilizar registros de propósito general y el registro SREG.

- Implementar saltos condicionales y estructuras de control.
- Implementar retardos mediante software.
- Leer pulsadores, sensores e interruptores digitales.
- Controlar LEDs, motores, una válvula y un display de siete segmentos.
- Aplicar operaciones lógicas, aritméticas, desplazamientos y máscaras.
- Verificar los programas mediante simulación y montaje físico.

III. MATERIALES

Los materiales utilizados durante el laboratorio son:

- Placa Arduino UNO con microcontrolador ATmega328P.
- Protoboard.
- Fuente de alimentación mediante USB o 5 V.
- LEDs.
- Resistencias limitadoras de corriente.
- Pulsadores.
- DIP switches.
- Display de siete segmentos.
- Cables de conexión.

IV. METODOLOGÍA GENERAL

Para cada actividad se siguió el siguiente procedimiento:

1. Análisis del enunciado.
2. Identificación de entradas y salidas.
3. Diseño de la estructura lógica o máquina de estados.
4. Asignación de registros y puertos del ATmega328P.
5. Implementación del programa en lenguaje ensamblador.
6. Compilación del programa.
7. Simulación o verificación lógica.
8. Implementación física sobre protoboard.
9. Pruebas experimentales.
10. Registro de modificaciones significativas en GitHub.

V. PROBLEMA A: LAVADORA AUTOMÁTICA

V-A. Descripción

El primer problema consiste en controlar una lavadora automática que dispone de tres tipos de carga: ligera, media y pesada. El sistema debe informar el estado actual mediante LEDs y controlar el llenado, el lavado, el centrifugado y el secado.

Las entradas principales son un pulsador de inicio, un pulsador de selección de carga, un sensor de puerta y un sensor de llenado.

Las salidas principales son los LEDs de indicación, el motor y la válvula de llenado.

V-B. Selección de carga

Para compactar la solución se utiliza una variable *Carga*, almacenada en un registro:

Cuadro I
CODIFICACIÓN DE LA CARGA

Valor	Tipo de carga
0	Ligera
1	Media
2	Pesada

La selección se mantiene durante todo el ciclo y se reinicia al regresar al estado *Stand By*.

V-C. Tiempos del proceso

Cuadro II
TIEMPOS SEGÚN LA CARGA SELECCIONADA

Carga	Tipo	Giro	Pausa	Centrif.	Secado D/E/I
0	Ligera	2 s	1 s	15 s	5/3/5 s
1	Media	3 s	2 s	18 s	7/5/7 s
2	Pesada	4 s	3 s	21 s	9/7/9 s

Los tiempos pueden expresarse mediante:

$$\begin{aligned}
 T_{giro} &= 2 + Carga & (1) \\
 T_{pausa} &= 1 + Carga & (2) \\
 T_{centrifugado} &= 15 + 3 \cdot Carga & (3) \\
 T_{secado,D} &= 5 + 2 \cdot Carga & (4) \\
 T_{secado,E} &= 3 + 2 \cdot Carga & (5) \\
 T_{secado,I} &= 5 + 2 \cdot Carga & (6)
 \end{aligned}$$

El ciclo giro-pausa del lavado se repite cinco veces.

V-D. Control de llenado

La válvula se activa únicamente durante el estado de llenado.

La válvula permanece abierta mientras el sistema se encuentra en el estado de llenado. El sensor de llenado se utiliza como condición de finalización de esta etapa: cuando detecta que se alcanzó el nivel requerido, la válvula se cierra y el sistema avanza al estado de lavado.

En la implementación final el llenado depende directamente del sensor y no de un retardo fijo, evitando continuar cargando agua una vez alcanzado el nivel deseado.

V-E. Máquina de estados

La secuencia compacta empleada es:

Stand By → Selección → Llenado → Lavado
Giro → Lavado Pausa → Centrifugado →
Secado D → Espera → Secado I → Fin

[Acceso a la carpeta del diagrama de estados de la lavadora](#)

Figura 1. Enlace a la documentación y diagrama de estados del Problema A en GitHub.

[Acceso al video demostrativo de la lavadora automática](#)

Figura 2. Enlace de acceso al video demostrativo del montaje físico de la lavadora automática.

V-F. Descripción de estados

Stand By: mantiene encendido el LED *Listo para lavar* y todas las salidas de potencia apagadas.

Selección de carga: se registra la carga ligera, media o pesada mediante un único pulsador.

Llenado: se activa la válvula mientras los motores permanecen apagados. La puerta debe estar cerrada antes de iniciar el proceso.

Lavado Giro: el motor gira durante el tiempo definido por la carga.

Lavado Pausa: el motor se detiene. Los estados de giro y pausa se repiten cinco veces.

Centrifugado: el motor gira a máxima velocidad durante 15, 18 o 21 s.

Secado Derecha: el motor gira hacia la derecha.

Secado en espera: ambos sentidos del motor permanecen apagados.

Secado Izquierda: el motor gira hacia la izquierda.

Fin del proceso: se enciende el LED de fin durante 2 s y posteriormente el sistema vuelve a *Stand By*.

V-G. Cálculo de las resistencias de los LEDs

Los LEDs utilizados para indicar los diferentes estados de la lavadora se conectaron mediante resistencias limitadoras de corriente de 330 Ω.

Aplicando la Ley de Ohm:

$$I_{LED} = \frac{V_{CC} - V_{LED}}{R}$$

Considerando una tensión de alimentación de:

$$V_{CC} = 5V$$

una caída de tensión aproximada en el LED de:

$$V_{LED} \approx 2V$$

y una resistencia de:

$$R = 330 \Omega$$

se obtiene:

$$I_{LED} = \frac{5 - 2}{330}$$

$$I_{LED} = 0.0091 A$$

$$I_{LED} \approx 9.1 mA$$

Por lo tanto, se utilizaron resistencias de 330Ω en serie con los LEDs indicadores de la lavadora, limitando la corriente a aproximadamente $9.1 mA$.

V-H. Código ensamblador

[Acceso al código ASM de la lavadora automática en GitHub](#)

Figura 3. Enlace al código fuente del Problema A disponible en el repositorio GitHub.

V-I. Resultados y evidencia

[Acceso a la simulación y captura de la lavadora automática](#)

Figura 4. Enlace a la carpeta de GitHub que contiene la simulación en Proteus y la captura de funcionamiento de la lavadora automática.

VI. PROBLEMA B: CONTADOR DIGITAL CON TRES BOTONES Y DISPLAY DE 7 SEGMENTOS

VI-A. Descripción

El segundo problema consiste en implementar un contador digital utilizando tres pulsadores y un display de siete segmentos.

El primer pulsador incrementa el contador, el segundo lo decrementa y el tercero reinicia el valor al estado inicial.

El valor del contador debe visualizarse permanentemente en el display.

VI-B. Entradas y salida

Cuadro III
ENTRADAS Y SALIDA DEL CONTADOR

Elemento	Función
Pulsador 1	Incrementar contador
Pulsador 2	Decrementar contador
Pulsador 3	Reiniciar contador
Display 7 segmentos	Mostrar valor actual

VI-C. Cálculo de las resistencias del display de 7 segmentos

Cada segmento del display está formado internamente por un LED, por lo que se utilizó una resistencia limitadora de corriente de 220Ω en serie con cada segmento.

Aplicando la Ley de Ohm:

$$I_{seg} = \frac{V_{CC} - V_{seg}}{R}$$

Considerando:

$$V_{CC} = 5 V$$

$$V_{seg} \approx 2 V$$

y:

$$R = 220 \Omega$$

se obtiene:

$$I_{seg} = \frac{5 - 2}{220}$$

$$I_{seg} = 0.0136 A$$

$$I_{seg} \approx 13.6 mA$$

Por lo tanto, se utilizaron resistencias de 220Ω en serie con cada segmento del display, limitando la corriente a aproximadamente $13.6 mA$. Cada segmento dispone de su propia resistencia limitadora.

VI-D. Estructura del programa

La implementación se divide en las siguientes tareas:

1. Configuración de los pulsadores como entradas.
2. Configuración de las líneas del display como salidas.
3. Lectura de los tres pulsadores.
4. Eliminación del rebote mediante una rutina de retardo.
5. Incremento, decremento o reinicio del registro contador.
6. Conversión del valor del contador al patrón de segmentos correspondiente.
7. Actualización permanente del display.

VI-E. Diagrama lógico

[Acceso al diagrama de estados del Problema B](#)

Figura 5. Enlace a la carpeta de GitHub que contiene el diagrama de estados del contador digital.

VI-F. Código ensamblador

[Acceso al código ASM del contador digital en GitHub](#)

Figura 6. Enlace al código fuente del Problema B disponible en el repositorio GitHub.

VI-G. Resultados y evidencia

Se verificó que el contador responde correctamente a los tres pulsadores: incremento, decremento y reinicio. El valor actual se visualiza de forma permanente en el display de siete

segmentos y la rutina de antirrebote evita múltiples conteos producidos por una única pulsación.

[Acceso al video demostrativo del contador digital](#)

Figura 7. Enlace al video demostrativo del montaje físico del contador digital.

[Acceso a la simulación y captura del contador digital](#)

Figura 8. Enlace a la carpeta de GitHub que contiene la simulación en Proteus y la captura de funcionamiento del contador digital.

VII. PROBLEMA C: SELECCIÓN Y EJECUCIÓN DE SECUENCIAS

VII-A. Descripción

El tercer problema consiste en implementar un sistema de selección y ejecución de ocho secuencias luminosas diferentes utilizando ocho LEDs y tres pulsadores. El primer pulsador permite avanzar a la secuencia siguiente, el segundo volver a la secuencia anterior y el tercero regresar directamente a la secuencia 1.

Cada secuencia se ejecuta de manera continua hasta que el usuario selecciona otra. No se utiliza display para indicar la secuencia activa, por lo que el propio patrón luminoso constituye la indicación visual del estado del sistema.

VII-B. Asignación de entradas y salidas

Para el montaje físico se utilizaron los pines digitales D2–D9 como salidas para los ocho LEDs y D10–D12 como entradas para los tres pulsadores.

Cuadro IV
ASIGNACIÓN DE PINES DEL PROBLEMA C

Elemento	Arduino UNO	ATmega328P
LED 0	D2	PD2
LED 1	D3	PD3
LED 2	D4	PD4
LED 3	D5	PD5
LED 4	D6	PD6
LED 5	D7	PD7
LED 6	D8	PB0
LED 7	D9	PB1
Siguiente	D10	PB2
Anterior	D11	PB3
Volver a secuencia 1	D12	PB4

Los pulsadores se conectaron entre el pin correspondiente y GND. En el programa se habilitaron las resistencias *pull-up* internas de PB2, PB3 y PB4, por lo que no fue necesario utilizar resistencias externas para las entradas. De esta forma, un pulsador sin presionar se lee como nivel alto y al presionarlo la entrada pasa a nivel bajo.

VII-C. Cálculo de las resistencias de los LEDs

Cada LED se conectó a una salida del microcontrolador mediante una resistencia limitadora de corriente de $220\ \Omega$.

La corriente aproximada se calcula mediante la Ley de Ohm:

$$I_{LED} = \frac{V_{CC} - V_{LED}}{R}$$

Considerando:

$$V_{CC} = 5\ V$$

$$V_{LED} \approx 2\ V$$

$$R = 220\ \Omega$$

se obtiene:

$$I_{LED} = \frac{5 - 2}{220}$$

$$I_{LED} = 0.0136\ A$$

$$I_{LED} \approx 13.6\ mA$$

Por lo tanto, las resistencias de $220\ \Omega$ limitan la corriente de cada LED a aproximadamente $13.6\ mA$.

VII-D. Control mediante pulsadores

Cuadro V
FUNCIÓN DE LOS PULSADORES

Pulsador	Función
D10	Siguiente secuencia
D11	Secuencia anterior
D12	Volver a secuencia 1

La selección es circular. Al avanzar desde la secuencia 8 se vuelve a la secuencia 1 y, al retroceder desde la secuencia 1, se pasa a la secuencia 8. El pulsador conectado a D12 fuerza directamente el retorno a la primera secuencia.

Para evitar cambios múltiples provocados por el rebote mecánico de los pulsadores se implementó una rutina de antirrebote mediante retardos por software.

VII-E. Secuencias implementadas

El programa almacena internamente el número de secuencia mediante valores de 0 a 7. Para el usuario, estos valores corresponden a las secuencias 1 a 8.

Cuadro VI
SECUENCIAS LUMINOSAS IMPLEMENTADAS

Secuencia	Patrón principal
1	Un LED desplazándose de LED0 hacia LED7
2	Un LED desplazándose de LED7 hacia LED0
3	Alternancia 10101010 / 01010101
4	Llenado progresivo hasta 11111111
5	Dos LEDs separados: 00010001 desplazándose
6	Dos LEDs consecutivos desplazándose
7	Parpadeo simultáneo de los ocho LEDs
8	Alternancia entre 00001111 y 11110000

Los patrones se generan mediante operaciones de desplazamiento, máscaras, complemento de bits y saltos condicionales. Las instrucciones `LSL` y `LSR` se utilizan para desplazar los patrones, mientras que la instrucción `COM` permite alternar rápidamente entre patrones complementarios.

VII-F. Funcionamiento general del programa

La variable `secuencia`, almacenada en el registro `r17`, indica qué secuencia está activa. El registro `patron` contiene los ocho bits que representan el estado instantáneo de los LEDs.

El bucle principal ejecuta de forma continua las siguientes tareas:

1. comprobar el estado de los tres pulsadores;
2. mostrar el patrón actual en los ocho LEDs;
3. aplicar un retardo entre pasos;
4. calcular el siguiente patrón de la secuencia activa;
5. repetir el proceso.

Los seis bits menos significativos del patrón se escriben en `PD2–PD7`, mientras que los dos bits más significativos se envían a `PB0–PB1`. De esta forma, una única variable de ocho bits permite controlar los ocho LEDs distribuidos entre los puertos B y D.

VII-G. Diagrama lógico

[Acceso al diagrama del Problema C](#)

Figura 9. Enlace a la carpeta de GitHub que contiene el diagrama de selección y ejecución de las ocho secuencias luminosas.

VII-H. Código ensamblador

[Acceso al código ASM de las ocho secuencias en GitHub](#)

Figura 10. Enlace al código fuente del Problema C disponible en el repositorio GitHub.

VII-I. Compilación y carga del programa

El programa fue desarrollado y compilado en Microchip Studio seleccionando el ATmega328P como dispositivo. La compilación generó correctamente el archivo `Ejercicio C L1.hex`.

La carga del archivo hexadecimal hacia el Arduino UNO se realizó mediante `avrdude` utilizando el bootloader de la placa. Durante esta prueba la placa fue detectada en el puerto `COM5`. La escritura y la verificación de la memoria Flash finalizaron correctamente, registrándose 374 bytes escritos y verificados.

VII-J. Pruebas experimentales

Después de cargar el programa se verificaron experimentalmente las ocho secuencias luminosas. Los patrones observados fueron:

Cuadro VII
VERIFICACIÓN EXPERIMENTAL DE LAS SECUENCIAS

Sec.	Comportamiento observado	Resultado
1	LED desplazándose LED0 → LED7	Correcto
2	LED desplazándose LED7 → LED0	Correcto
3	LEDs alternados	Correcto
4	Llenado progresivo	Correcto
5	Dos LEDs separados desplazándose	Correcto
6	Dos LEDs consecutivos desplazándose	Correcto
7	Ocho LEDs parpadeando simultáneamente	Correcto
8	Alternancia de las dos mitades	Correcto

Durante las pruebas experimentales se verificó correctamente el funcionamiento de los tres pulsadores de control. El pulsador conectado a D10 permitió avanzar a la secuencia siguiente, el pulsador conectado a D11 permitió regresar a la secuencia anterior y el pulsador conectado a D12 permitió retornar directamente a la secuencia 1. También se comprobó la selección circular entre las secuencias 1 y 8 y el correcto funcionamiento del antirrebote y la espera de liberación.

VII-K. Resultados y evidencia

Las pruebas realizadas permitieron comprobar el funcionamiento de los ocho LEDs, la generación correcta de los ocho patrones, la actualización continua de cada secuencia y el funcionamiento de los tres pulsadores de control. Se verificaron el avance, el retroceso y el retorno directo a la secuencia 1. Todas las secuencias indicadas en la Tabla VI fueron observadas correctamente en el montaje físico.

[Acceso al video demostrativo de las 8 secuencias](#)

Figura 11. Enlace al video demostrativo del montaje físico de las ocho secuencias luminosas.

[Acceso a la simulación y captura de las secuencias luminosas](#)

Figura 12. Enlace a la carpeta de GitHub que contiene la simulación en Proteus y la captura de funcionamiento de las ocho secuencias luminosas.

VIII. PROBLEMA D: UNIDAD LÓGICA ARITMÉTICA (ALU)

VIII-A. Descripción

El cuarto problema consiste en diseñar e implementar una Unidad Aritmético-Lógica (ALU) de 4 bits utilizando el microcontrolador ATmega328P programado en lenguaje ensamblador.

La ALU trabaja con dos operandos de cuatro bits, denominados *A* y *B*, y una entrada de selección *S* de tres bits. Dependiendo del valor de $S_2S_1S_0$, el sistema ejecuta una de ocho operaciones aritméticas o lógicas.

Además del resultado $F_3F_2F_1F_0$, se implementan tres banderas de estado: Carry (C), Negativo (N) y Zero (Z). La operación seleccionada se identifica visualmente mediante tres LEDs asociados a las líneas S_0 , S_1 y S_2 .

VIII-B. Tabla de operaciones

Cuadro VIII
OPERACIONES IMPLEMENTADAS EN LA ALU

$S_2S_1S_0$	Operación	Resultado F
000	CLEAR	0000
001	$A - B$	$A - B$
010	$A + B$	$A + B$
011	XOR	$A \oplus B$
100	AND	$A \wedge B$
101	OR	$A \vee B$
110	SHL	$A \ll 1$
111	INC	$A + 1$

VIII-C. Simplificación del montaje

Durante una primera etapa de diseño se evaluó utilizar un registro de expansión de salidas. Sin embargo, el montaje definitivo se simplificó reorganizando por completo la asignación de GPIO del Arduino UNO.

El diseño final utiliza directamente dieciocho GPIO del ATmega328P y mantiene libres D0 y D1, correspondientes a RX y TX. Los tres LEDs encargados de indicar el código de selección se conectan sobre los mismos nodos de entrada S_0 , S_1 y S_2 , por lo que no requieren tres GPIO de salida adicionales.

De esta forma, el circuito definitivo no utiliza registro de desplazamiento y las siete salidas propias de la ALU se controlan directamente desde el microcontrolador.

VIII-D. Entradas de la ALU

Para establecer los operandos y seleccionar la operación se utilizan once interruptores tipo DIP switch:

- cuatro para el operando A ;
- cuatro para el operando B ;
- tres para el selector S .

Cada interruptor se conecta entre el pin correspondiente y GND. Las entradas utilizan las resistencias *pull-up* internas del ATmega328P. Como el nivel eléctrico producido al cerrar un DIP es cero, el programa invierte la lectura mediante la instrucción COM. De esta manera la interfaz queda más intuitiva:

DIP switch en ON = 1 lógico

DIP switch en OFF = 0 lógico

Esta inversión se aplica únicamente a la interpretación lógica de las entradas; eléctricamente, un DIP cerrado continúa conectando el pin a GND.

VIII-E. Asignación de entradas

Cuadro IX
ASIGNACIÓN DE ENTRADAS DE LA ALU

Señal	Arduino UNO	ATmega328P
A_0	A0	PC0
A_1	A1	PC1
A_2	A2	PC2
A_3	A3	PC3
B_0	D2	PD2
B_1	D3	PD3
B_2	D4	PD4
B_3	D5	PD5
S_0	D6	PD6
S_1	D7	PD7
S_2	D8	PB0

VIII-F. Indicadores del selector

Los tres LEDs de selección se conectan a los mismos nodos de S_0 , S_1 y S_2 . Cada indicador se alimenta desde 5 V mediante una resistencia limitadora y su cátodo se conecta al nodo correspondiente, el cual también se conecta al DIP y al pin del microcontrolador.

Cuando el DIP está en ON, el nodo se lleva a GND, el LED se enciende y el programa interpreta ese estado como un uno lógico. Cuando el DIP está en OFF, el nodo queda en nivel alto por la resistencia *pull-up*, el LED se apaga y el programa interpreta un cero lógico.

Cuadro X
ENTRADAS E INDICADORES DEL CÓDIGO DE SELECCIÓN

Bit	Arduino UNO	Indicador
S_0	D6	LED S_0
S_1	D7	LED S_1
S_2	D8	LED S_2

De esta manera el código de selección puede observarse directamente sin consumir GPIO adicionales de salida.

VIII-G. Cantidad de GPIO utilizados

Las entradas propias de la ALU requieren:

4 bits de A + 4 bits de B + 3 bits de S = 11 entradas

Las salidas propias requieren:

4 bits de F + C + N + Z = 7 salidas

Por lo tanto, el diseño final utiliza:

11 + 7 = 18 GPIO

La placa Arduino UNO dispone justamente de dieciocho GPIO utilizables si se reservan D0 y D1 para comunicación serie. Los tres LEDs de selección no agregan GPIO porque se encuentran asociados directamente a las entradas S_0 – S_2 .

VIII-H. Asignación de salidas

Las siete salidas se conectan directamente a LEDs mediante resistencias limitadoras de corriente.

Cuadro XI
ASIGNACIÓN DE SALIDAS DE LA ALU

Señal	Arduino UNO	ATmega328P
F_0	D9	PB1
F_1	D10	PB2
F_2	D11	PB3
F_3	D12	PB4
Carry (C)	D13	PB5
Negativo (N)	A4	PC4
Zero (Z)	A5	PC5

Cada salida utiliza la conexión:

GPIO → Resistencia → Ánodo LED
Cátodo LED → GND

El pin D13 también controla el LED integrado de la placa Arduino UNO; por este motivo, cuando Carry vale uno, puede encenderse simultáneamente el LED externo y el LED integrado de la placa.

VIII-I. Banderas de estado

La ALU genera tres banderas:

- **Carry (C):** indica acarreo en suma e incremento, préstamo en resta o desplazamiento fuera del rango en SHL.
- **Negativo (N):** se activa cuando el bit más significativo F_3 es igual a uno.
- **Zero (Z):** se activa cuando $F = 0000$.

En la suma y en INC se analiza temporalmente el bit 4 antes de limitar el resultado a cuatro bits. En la resta se detecta el préstamo cuando $A < B$. En SHL, Carry toma el valor previo de A_3 . En XOR, AND y OR, Carry se fuerza a cero.

VIII-J. Cálculo de las resistencias de los LEDs

Para limitar la corriente que circula por los LEDs se utilizaron resistencias de $1\text{ k}\Omega$.

El valor puede justificarse mediante la Ley de Ohm:

$$R = \frac{V_{CC} - V_{LED}}{I_{LED}}$$

Considerando una alimentación de:

$$V_{CC} = 5\text{ V}$$

y una caída de tensión aproximada en el LED de:

$$V_{LED} \approx 2\text{ V}$$

la corriente obtenida utilizando una resistencia de $1\text{ k}\Omega$ es:

$$I_{LED} = \frac{5 - 2}{1000}$$

$$I_{LED} = 0.003\text{ A}$$

$$I_{LED} \approx 3\text{ mA}$$

Por lo tanto, las resistencias de $1\text{ k}\Omega$ permiten limitar la corriente a aproximadamente 3 mA , valor suficiente para visualizar correctamente los LEDs y mantener una corriente reducida en las salidas del ATmega328P.

VIII-K. Funcionamiento general del programa

Después de configurar los puertos, el programa entra en un bucle continuo con las siguientes etapas:

1. lectura de A , B y S ;
2. inversión de las entradas para obtener $\text{ON}=1$;
3. selección de la operación;
4. cálculo del resultado F ;
5. cálculo de Carry;
6. actualización directa de F_0 – F_3 , C, N y Z;
7. repetición inmediata del ciclo.

El proceso puede resumirse mediante:

Leer entradas → Invertir → Calcular ALU
→ Actualizar salidas → Repetir

Debido a que el proceso se ejecuta continuamente, cualquier modificación de los DIP switches se refleja inmediatamente en los LEDs.

VIII-L. Implementación de las operaciones

La selección se realiza mediante comparaciones sucesivas del registro S . Las instrucciones principales son:

- ADD: suma;
- SUB: resta;
- EOR: XOR;
- AND: AND;
- OR: OR;
- LSL: desplazamiento a izquierda;
- CPI: comparación;
- BREQ: salto si los valores comparados son iguales.

Como la ALU es de cuatro bits, el resultado final se limita mediante:

ANDI F, 0x0F

La máscara $0x0F = 00001111_2$ conserva solamente $F_3F_2F_1F_0$.

VIII-M. Actualización directa de las salidas

La rutina MOSTRAR_SALIDA genera directamente los niveles de los puertos B y C. En PORTB se preserva PB0 como entrada con *pull-up* y se colocan F_0 , F_1 , F_2 , F_3 y Carry en PB1–PB5. En PORTC se preservan PC0–PC3 como entradas con *pull-up* y se utilizan PC4 y PC5 para N y Z.

Esta implementación elimina la transmisión serie utilizada en la versión anterior del montaje y reduce la cantidad de rutinas necesarias.

VIII-N. Diagrama de flujo de la ALU

El funcionamiento general del programa se representa mediante un diagrama de flujo en el cual se muestran la configuración inicial, la lectura e inversión de los operandos A y B , la evaluación del selector $S_2S_1S_0$, la ejecución de las ocho operaciones, el cálculo de las banderas y la actualización directa de las salidas.

A diferencia de una máquina de estados secuencial, la ALU funciona mediante un ciclo continuo. Una vez actualizado el resultado, el programa vuelve a leer las entradas.

[Acceso al diagrama de flujo de la ALU de 4 bits](#)

Figura 13. Enlace de acceso al diagrama de flujo de funcionamiento de la ALU.

VIII-Ñ. Código ensamblador

[Acceso al código ASM de la ALU de 4 bits en GitHub](#)

Figura 14. Enlace al código fuente del Problema D disponible en el repositorio GitHub.

VIII-O. Compilación y carga del programa

El programa de la ALU fue desarrollado en lenguaje ensamblador utilizando Microchip Studio y seleccionando como dispositivo el ATmega328P. La compilación se realizó mediante *Build Solution*, generándose el archivo hexadecimal .hex.

La transferencia hacia la placa Arduino UNO se realizó mediante el bootloader y la herramienta *avrdude*. Después de comprobar el puerto asignado por Windows, la programación se realizó correctamente mediante COM6, verificándose también el contenido escrito en la memoria Flash.

VIII-P. Verificación individual de las salidas

Durante la puesta en marcha se realizaron programas mínimos para comprobar cada LED de resultado de forma independiente. Se verificó la correspondencia:

$$D9=F0, D10=F1, D11=F2, D12=F3$$

Cada prueba configuró solamente un bit de PORTB como salida en nivel alto. De esta manera se pudo confirmar el cableado físico antes de volver a cargar el programa completo de la ALU.

VIII-Q. Prueba inicial de funcionamiento

Con el programa definitivo, la primera prueba se realizó dejando todos los DIP switches en OFF. Debido a la inversión por software:

$$\text{DIP OFF} = 0$$

Por lo tanto:

$$A = 0000_2 \quad (7)$$

$$B = 0000_2 \quad (8)$$

$$S = 000_2 \quad (9)$$

La operación seleccionada es CLEAR, por lo que:

$$F = 0000, \quad C = 0, \quad N = 0, \quad Z = 1$$

Experimentalmente se verificó que los cuatro LEDs de resultado permanecían apagados y que únicamente la bandera Zero se encontraba encendida. Los tres LEDs del selector permanecían apagados, indicando $S = 000$.

VIII-R. Registro de pruebas

Para verificar completamente la ALU se definieron los siguientes casos de prueba. Los valores de la tabla se expresan en orden lógico $A_3A_2A_1A_0$, $B_3B_2B_1B_0$ y $F_3F_2F_1F_0$.

Cuadro XII
CASOS DE PRUEBA DE LA ALU

S	Operación	A	B	F	C	N	Z
000	CLEAR	0000	0000	0000	0	0	1
001	$A - B$	0011	0101	1110	1	1	0
010	$A + B$	0010	0001	0011	0	0	0
011	XOR	1100	1010	0110	0	0	0
100	AND	1101	1011	1001	0	1	0
101	OR	1001	0100	1101	0	1	0
110	SHL	1001	–	0010	1	0	0
111	INC	1111	–	0000	1	0	1

Durante las comprobaciones del montaje se verificaron correctamente las ocho operaciones implementadas en la ALU: CLEAR, resta, suma, XOR, AND, OR, SHL e INC. En todos los casos el resultado obtenido coincidió con el valor esperado y las banderas Carry (C), Negativo (N) y Zero (Z) respondieron correctamente.

También se comprobó una resta con $A < B$, verificándose la activación de la bandera Carry para indicar préstamo. Para la operación INC se utilizó el caso $A = 1111$ y $S = 111$, obteniéndose $F = 0000$, $C = 1$, $N = 0$ y $Z = 1$, tal como se esperaba para una ALU de 4 bits.

VIII-S. Resultados y evidencia

[Acceso al video demostrativo de la ALU](#)

Figura 15. Enlace al video demostrativo del funcionamiento físico de la ALU de 4 bits.

[Acceso a la simulación y captura de la ALU](#)

Figura 16. Enlace a la carpeta de GitHub que contiene la simulación en Proteus y la captura de funcionamiento de la ALU de 4 bits.

IX. RESUMEN DE EVIDENCIAS

Cuadro XIII
EVIDENCIAS DE FUNCIONAMIENTO Y SIMULACIÓN

Problema	Video demostrativo	Simulación
A – Lavadora	Incluido	Enlace incluido
B – Contador	Incluido	Enlace incluido
C – Secuencias	Enlace incluido	Enlace incluido
D – ALU	Enlace incluido	Enlace incluido

X. GESTIÓN DEL CÓDIGO CON GITHUB

Además del acceso general al repositorio, en la subsección de código de cada problema se incorpora un enlace directo a la carpeta correspondiente en GitHub. De esta forma, el código fuente puede consultarse incluso cuando el archivo `.asm` no se encuentra cargado dentro del proyecto de Overleaf.

[Acceso al repositorio de GitHub del Laboratorio 1](#)

X-A. Trazabilidad de modificaciones significativas

Durante el desarrollo se registraron en GitHub distintas versiones del código. Entre las modificaciones más relevantes se destacan:

1. reorganización y actualización de los códigos de los cuatro ejercicios;
2. incorporación y corrección del sistema de ocho secuencias luminosas;
3. simplificación de la ALU, eliminando el registro de desplazamiento externo y reasignando los GPIO para conectar directamente resultados y banderas;
4. correcciones posteriores del código de la lavadora y de su organización dentro del repositorio.

Estas modificaciones permiten mantener trazabilidad entre las versiones iniciales, las correcciones realizadas durante las pruebas y las implementaciones finales.

XI. RESULTADOS GENERALES

Las cuatro actividades permiten verificar diferentes recursos de la arquitectura AVR.

En la lavadora automática se aplica una máquina de estados con sensores, temporizaciones y salidas de control. En el contador digital se emplea lectura de pulsadores, antirrebote y conversión hacia un display de siete segmentos. En el sistema de secuencias se aplican desplazamientos, máscaras, registros, saltos condicionales y ocho patrones luminosos diferentes. El montaje físico utiliza ocho LEDs conectados entre D2 y D9 mediante resistencias de 220 Ω y tres pulsadores en D10–D12 con resistencias *pull-up* internas.

Finalmente, la ALU integra operaciones aritméticas y lógicas, lectura de entradas binarias, generación de banderas y visualización mediante LEDs.

Durante el desarrollo físico de la ALU se realizó una modificación importante del diseño. La primera propuesta utilizaba expansión de salidas, mientras que la versión definitiva reorganiza los GPIO para conectar directamente las siete

salidas propias de la ALU al ATmega328P. Los tres LEDs de selección se asocian a los propios nodos S_0 – S_2 , por lo que no requieren pines de salida adicionales.

El montaje final utiliza dieciocho GPIO y mantiene D0 y D1 libres para la comunicación serie. Los DIP switches permiten mantener de forma estable los operandos A , B y el selector S . La inversión de las entradas por software permite utilizar una interfaz intuitiva en la cual ON representa uno lógico y OFF representa cero lógico.

Las pruebas individuales de D9, D10, D11 y D12 permitieron confirmar el mapeo de F_0 a F_3 . Posteriormente se verificaron operaciones lógicas y aritméticas sobre el montaje completo, incluyendo suma, resta, XOR, AND, OR y SHL, comprobando también el comportamiento de las banderas C, N y Z.

XII. CONCLUSIONES

El Laboratorio 1 permite integrar conceptos fundamentales del microcontrolador ATmega328P mediante diferentes aplicaciones prácticas y programación en lenguaje ensamblador.

La actividad de la lavadora demuestra la utilidad de las máquinas de estados para organizar procesos secuenciales con sensores y temporizaciones. El contador digital refuerza el manejo de entradas mediante pulsadores y la codificación de salidas hacia un display de siete segmentos. El sistema de ocho secuencias permite aplicar desplazamientos, máscaras, saltos condicionales y rutinas modulares. Las ocho secuencias fueron verificadas experimentalmente sobre el montaje físico.

La ALU permite integrar operaciones aritméticas y lógicas sobre datos binarios de cuatro bits y analizar las condiciones de Carry, Negativo y Zero. El uso de DIP switches resultó adecuado para fijar palabras binarias estables durante las pruebas.

Una modificación relevante fue la simplificación del circuito de la ALU. La versión definitiva eliminó el registro de expansión de salidas y reorganizó el pinout para utilizar directamente dieciocho GPIO del Arduino UNO, manteniendo D0 y D1 libres. Los bits F_0 – F_3 y las banderas C, N y Z se conectan de forma directa al ATmega328P, mientras que los LEDs del selector comparten los nodos de entrada S_0 – S_2 .

También se modificó la interpretación de los DIP switches. Aunque eléctricamente un DIP cerrado conecta la entrada a GND, el programa invierte el valor leído mediante COM, logrando que ON represente uno lógico y OFF represente cero lógico. Esta decisión facilita la configuración y explicación de los operandos y del código de operación.

Las pruebas individuales de los puertos permitieron confirmar la asignación física de las salidas antes de realizar las pruebas funcionales. Posteriormente se verificó experimentalmente el correcto funcionamiento de las ocho operaciones de la ALU: CLEAR, resta, suma, XOR, AND, OR, SHL e INC, junto con la respuesta correspondiente de las banderas C, N y Z.

Finalmente, las actividades desarrolladas permitieron comprobar el proceso de compilación y programación del ATmega328P, la configuración de GPIO, la lectura de entradas,

el uso de operaciones de bajo nivel y la visualización de resultados mediante periféricos digitales.

REFERENCIAS

- [1] Microchip Technology Inc., *ATmega328P 8-bit AVR Microcontroller Datasheet*.
- [2] UTEC, Ingeniería Mecatrónica, *Laboratorio 1 – GPIO, Tecnologías de Microprocesamiento*.